IACR Transactions on Cryptographic Hardware and Embedded Systems
ISSN 2569-2925, Vol. 2026, No. 2, pp. 709–735. DOI:10.46586/tches.v2026.i2.709-735

NTT-PEEL: Bit Shift Side-Channel in FALCON's Number Theoretic Transform

Jinyi Qiu and Aydin Aysu

Department of Electrical and Computer Engineering
North Carolina State University
{jqiu2,aaysu}@ncsu.edu

Abstract. This work introduces a new *single-trace* side-channel attack against FALCON, a lattice-based post-quantum digital signature scheme recently selected for standardization by NIST. The proposed attacks exploit a new vulnerability in the *in-place modular reduction* within the number theoretic transform (NTT). The vulnerable component executes both in the key generation and signature generation schemes of FALCON. Our key innovation is a custom attack method, which consists of forward and backward pass algorithms that combine partial leakages by exploiting the inherent structural dependencies. Overall, the method serves as an efficient mechanism for *peeling* key-dependent information from the NTT. We demonstrate the attack on an ARM Cortex-M4 microcontroller, evaluating both the reference and optimized implementations from FALCON's NIST round 3 software package. We quantified that our attack reduces the search space to 2^{24} with the assistance of lattice reduction algorithms. These findings expose the susceptibility of current software deployments to single-trace exploitation and emphasize the pressing need for embedded implementations hardened against such attacks.

Keywords: FALCON · Side channel · Post-quantum cryptography · Signature generation · Key generation · Lattice cryptography · Number theoretic transform

1 Introduction

The advancement of quantum computing technologies poses a direct and severe threat to traditional public-key encryption. Indeed, the mathematical problems adopted by the conventional schemes [RSA78, Kob87], which were traditionally considered intractable, are solved with exponential speed up with quantum computers. In response, the National Institute of Standards and Technology (NIST) launched a standardization effort to protect sensitive information from quantum cryptanalysis [CCJ+16].

NIST selected three digital signature schemes for standardization: CRYSTALS-Dilithium [LDK+20], SPHINCS+ [BHK+19], and FALCON [PFH+20]. Built upon lattice-based cryptography, FALCON employs the Gentry-Peikert-Vaikuntanathan (GPV) framework [DLP14] that offers security guarantees. FALCON also extensively bases its computations on the fast Fourier sampling algorithm, which improves the efficiency of the underlying arithmetic computations. It also stands out for its small signature size and rapid verification. These characteristics make it particularly well-suited for embedded platforms, mobile devices, and other environments where resource is limited.

Unfortunately, the real-world implementations of the selected algorithms can still be susceptible to side-channel attacks. These attacks exploit physical leakages related to implementation characteristics instead of cryptography defects [KJJ99]. Examples of such characteristics are: variations in execution time [DH96], power usage [KJJ99], or electromagnetic emmitance [KA98]. In many cases, the adversary does not require a

Licensed under Creative Commons License CC-BY 4.0.
Received: 2025-10-15 Accepted: 2025-12-15 Published: 2026-04-23

large number of measurements from the target device to compromise security [RRCB20, MKAA24, KDYA25]. The most dangerous variant, *single-trace* attacks (also referred to as simple power analysis), enables an adversary to recover sensitive data from just one program execution. These attacks are particularly dangerous because they bypass common defenses such as masking [NDGJ21] and make attacks possible on the leakages of nonces and other one-time variables. Given FALCON's suitability for embedded platforms, it becomes an especially attractive target for such exploitation. With NIST's standards on the verge of deployment [SY23, LLJ+24, LW23], uncovering these vulnerabilities and informing the design of robust countermeasures is both urgent and essential.

Related work Lattice-based cryptosystems contain unique components that are vulnerable to *single-trace* side-channel attacks. Previous studies demonstrated the following vulnerable components: polynomial multiplication [ATT+18a, BFM+18], message encoding/decoding [SKL+20, RBL24], cumulative distribution table (CDT) sampling [KH18, CHH24], discrete Gaussian sampling [KH18, ZLC+20, QA25], and the Fujisaki-Okamoto transform [JNWD24]. Although FALCON incorporates some of these components, its implementation characteristics remain different from the ones above. Therefore, existing attacks and defenses cannot be directly applied to these FALCON-specific operations, highlighting the need for further investigation into vulnerabilities specific to FALCON.

Recent works have revealed both multi-trace and single-trace side-channel vulnerabilities in FALCON. Among multi-trace attacks, Karabulut et al. [KA21] conducted the first side-channel evaluation of the scheme by targeting its point-wise multiplication. Guerreau et al. [GMRR22] advanced this approach by exploiting FALCON's integer coefficient structure and redundant complex multiplications. Building on these insights, Zhang et al. [ZLYW23] leveraged second-order covariance statistics and spectral decomposition to enhance signal extraction, while Lin et al. [LZY+25] further reduced the required number of traces by exploiting the symplectic properties of NTRU. More recently, Chen et al. attacked FALCON's floating-point multiplications during the FFT operations [CSM+25]. In addition, Lin et al. [LTYZ25] identified precision-related leakages in the floating-point arithmetic units that may expose secret data through repeated invocations. Mask-able variants of FALCON, including Mitaka [EFG+22], have also been shown to remain susceptible to side-channel exploitation [SD25]. FALCON's vulnerability to single-trace attacks has also been established. Qiu et al. [QA25] demonstrated information leakage in the discrete Gaussian sampler, and Choi et al. [CHH24] revealed exploitable leakage in the cumulative distribution table sampling process. Guerreau et al. [GMRR22] also demonstrated single trace analysis components. Attacks targeting the GPV signing component have been shown to recover the secret key when combined with advanced techniques such as non-filtering secret-key recovery [NCHS24].

Our contributions In this paper, we present a *novel* single-trace side-channel attack on FALCON that is distinct from previously reported attacks [CHH24, GMRR22, ZLYW23, LZY+25, QA25]. The contributions of this paper are as follows:

- We uncover a previously undocumented side-channel leakage in FALCON's Number Theoretic Transform (NTT) that enables recovery of the secret key from a *single* power trace. The proposed attack exploits the in-place modular reduction within the NTT subroutine and leaks information about whether the modular reduction occurred. We demonstrate that the vulnerable NTT subroutine is called both during the key generation and signature generation processes of FALCON, both of which lead to a search space sufficient for a brute-force attack.

- We introduce a custom key recovery algorithm that consists of a novel forward and backward passing over the NTT for key recovery. The proposed algorithm efficiently

combines partial leakages by exploiting the inherent structural dependencies within each stage of the NTT. Unlike previous methods[PPM17, PP19], our approach provides deterministic guidance on key guesses and achieves an average 50% candidate elimination from a single butterfly operation. We optimized our implementation of the proposed algorithm using an NVIDIA RTX-3090 GPU. Compared to the standard CPU implementation, the optimized version offers at least 3450× speed up for key recovery. To the best of our knowledge, this is the *first* work to demonstrate this deterministic message passing method for key recovery in lattice-based schemes.

- We validate the attack on a commercially available ARM Cortex-M4 microcontroller, one of the most widely used evaluation platforms for side-channel analysis. We demonstrate the attack on both reference optimized implementations running on this platform. Our results show that, with the help of lattice reduction techniques, the search space is efficiently reduced to 2^{24}, which makes a brute-force attack feasible.

Organization The rest of this paper is organized as follows. Section 2 provides the necessary background on FALCON and the Number Theoretic Transform (NTT), situating our work within the context of existing research and clarifying the specific gap our attack In Section 3, we conduct a detailed foundational analysis, pinpointing a critical leakage source in FALCON's NTT subroutine and demonstrating how this vulnerability can be systematically leveraged for high-confidence key recovery. Section 4 outlines our attack framework, which integrates a guess-and-prune (GP) strategy to achieve single-trace key reconstruction. Our experimental evaluation, presented in Section 5, validates the effectiveness of the attack, demonstrates substantial performance improvements through GPU acceleration, and provides a detailed analysis of FALCON's brute-force search space before and after the attack. In Section 6, we discuss defense methods, and the broader security implications of our findings for the post-quantum cryptography landscape. Finally, Section 7 recaps our main contributions and emphasizes the pressing need for single-trace-resilient implementations as post-quantum standards transition into widespread use.

2 Background

This section provides extensive background information about our attack. We begin by providing an overview of FALCON and its secret keys, which our attack can recover. Then, we explain the number theoretic transform (NTT) operation and its usage in FALCON's implementation. We provide background on belief propagation (BP), which is a widely used method to attack the NTT. Finally, we present our threat model.

FALCON's secret polynomials FALCON (Fast Fourier Lattice-based Compact Signature over NTRU) is a post-quantum digital signature scheme engineered to remain secure even in the presence of quantum-capable adversaries. Similar to other digital signature schemes, FALCON is composed by three stages: key generation, signature generation, and signature verification. Its security relies on the intractability of the Shortest Integer Solution (SIS) problem, instantiated within the NTRU lattice framework, which remains computationally prohibitive even for quantum algorithms. Compared to many other post-quantum signature schemes, FALCON achieves notably compact key sizes and signatures, enabling high efficiency and making it particularly well-suited for deployment in environments with strict resource limitations.

As shown in Algorithm 1, the first step of FALCON signature scheme is to generate the base NTRU lattice polynomials f and g with a fixed parameter n (either 512 or 1024) and a modulus q (set to 12289 in the NIST submission package). From here, both the

Algorithm 1 FALCON Secret Generation

Require: A monic polynomial $\phi \in \mathbb{Z}[x]$, a modulus q
Ensure: A secret key sk, a public key pk

1: $f, g, F, G \leftarrow \text{NTRUGen}(\phi, q)$ ▷ Solving the NTRU equation
2: $\mathbf{B} \leftarrow \begin{bmatrix} g & -f \\ G & -F \end{bmatrix}$
3: $\hat{\mathbf{B}} \leftarrow \text{FFT}(\mathbf{B})$ ▷ Compute the FFT for each of the 4 components $\{g, -f, G, -F\}$
4: $\mathbf{G} \leftarrow \hat{\mathbf{B}} \times \hat{\mathbf{B}}^{\star}$
5: $\mathbf{T} \leftarrow \text{ffLDL}^{\star}(\mathbf{G})$ ▷ Computing the LDL* tree
6: **for each** leaf of $\mathbf{T}$ **do** ▷ Loop over the leaves of $\mathbf{T}$
7: leaf.value $\leftarrow \sigma/\sqrt{\text{leaf.value}}$ ▷ Normalization step
8: sk $\leftarrow (\hat{\mathbf{B}}, \mathbf{T})$
9: $h \leftarrow gf^{-1} \mod q$
10: pk $\leftarrow h$
11: **return** sk, pk

public and secret keys are deterministically derived without any additional randomness or auxiliary secret material. This property makes f and g *highly security-critical*, as the entire key pair is fixed once these polynomials are produced. The public key h is computed from f and g and the derivation is infeasible to invert.

As shown in line 1 of Algorithm 1, the `NTRUGen()` function constitutes the initial step in producing the base NTRU lattice polynomials f and g. It specifies f and g as degree-n polynomials, adhering to the structure outlined in the equations below:

$$f(x) = f_0 + f_1 x + f_2 x^2 + f_3 x^3 + \cdots + f_{n-1} x^{n-1} \tag{1}$$

$$g(x) = g_0 + g_1 x + g_2 x^2 + g_3 x^3 + \cdots + g_{n-1} x^{n-1} \tag{2}$$

In the above formulations, the coefficients of f and g are drawn from a discrete Gaussian distribution with mean zero and a standard deviation determined by the parameters n and q. For FALCON-512, $n = 512$, the modulus is fixed at q=12289.

NTT Let q be a prime such that $N = 2^m \mid (q-1)$, ensuring the existence of a primitive N-th root of unity $\omega \in \mathbb{Z}_q^{\times}$ satisfying $\omega^N \equiv 1 \pmod q$ and $\omega^k \not\equiv 1 \pmod q$ for $1 \le k < N$. For a vector $\mathbf{x} = (x_0, \ldots, x_{N-1}) \in \mathbb{Z}_q^N$, the *forward NTT* is defined as

$$X_k = \sum_{n=0}^{N-1} x_n \, \omega^{nk} \pmod q, \qquad 0 \le k < N, \tag{3}$$

and the *inverse transform* is

$$x_n = N^{-1} \sum_{k=0}^{N-1} X_k \, \omega^{-nk} \pmod q, \tag{4}$$

where $N^{-1} \equiv N^{q-2} \pmod q$. The NTT establishes the ring isomorphism

$$\mathbb{Z}_q[X]/(X^N - 1) \;\cong\; \mathbb{Z}_q^N, \tag{5}$$

so that for polynomials $A(X), B(X) \in \mathbb{Z}_q[X]$,

$$\text{NTT}^{-1}\big(\text{NTT}(A) \odot \text{NTT}(B)\big) \equiv AB \pmod{X^N - 1, q}, \tag{6}$$

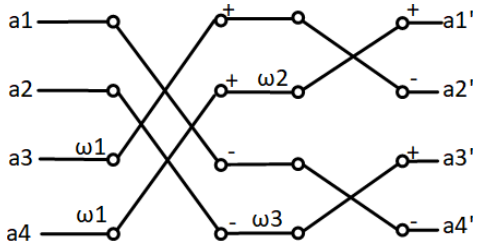


Figure 1: A simplified 2-stage butterfly structure of FALCON's NTT implementation that followed the decimation in frequency method. FALCON's NTT implementation combines single butterfly operations that obey the decimation-in-frequency method. The nth stage has 2^n groups, all butterflies within the same group share an identical twiddle factor.

enabling fast modular convolution. For $N = 2^m$, the transform admits a radix-2 Cooley-Tukey factorization. At each stage s, butterflies compute

$$\begin{aligned} u = a_{k+j}, \quad v = a_{k+j+2^{s-1}}\, \omega^{j2^{m-s}}, \\ a_{k+j} \leftarrow u + v \pmod q, \quad a_{k+j+2^{s-1}} \leftarrow u - v \pmod q, \end{aligned} \tag{7}$$

forming a structured network expressible as

$$\mathbf{X} = \Pi \prod_{s=1}^{m} \left(I_{N/2^s} \otimes (B_s D_s)\right)\mathbf{x}, \tag{8}$$

where Π is the bit-reversal permutation, B_s denotes the butterfly matrix, and D_s the twiddle-factor diagonal. This formulation reduces polynomial multiplication from $\mathcal{O}(N^2)$ to $\mathcal{O}(N \log N)$ under modular arithmetic.

FALCON's NTT variant Figure 1 offers a simplified 2-stage demonstration of FALCON's NTT implementation. Although not explicitly defined in the FALCON specification document, the corresponding NTT software implementations exhibit the following characteristics. The NTT's butterfly operations are specifically tailored for its prime modulus $q = 12289$ and use precomputed twiddle factors in the Montgomery domain to improve computational efficiency. Within the same stage of NTT operation, the butterflies are separated into 2^n groups; butterflies in the same group share the same twiddle factor. The NTT implementation in FALCON follows the decimation-in-frequency (DIF) variant of the Cooley-Tukey decomposition, meaning that stride lengths are processed in reverse order and the explicit rearrangement of data indices (bit reversal) is omitted. Because each butterfly operation acts only on a pair of elements, the overall algorithm is highly parallelizable and well suited for hardware acceleration. We take a deeper dive on FALCON's NTT implementation characteristics on the code level in Section 3.

Belief propagation Belief propagation (BP) is widely adopted in side-channel attacks targeting the number theoretic transform (NTT). It is an efficient method for computing marginal probabilities in probabilistic graphical models, particularly factor graphs. BP operates by passing two types of messages, variable-to-factor and factor-to-variable, along the edges of the graph, with variable nodes being iteratively updated through local computations. This approach was first introduced by Charvillon et al. as part of the soft analytical side-channel attack (SASCA) method in 2014 [VCGS14]. The first study targeting NTT single-trace vulnerabilities, conducted by Primas et al., employed this method [PPM17]. Pessl et al. subsequently proposed improvements to this attack [PP19], making it more tolerant to noise and reducing the required attack effort. Further refinements were presented by Hermelink, who conducted an attack on a scheme with shuffled computations [HSST23],

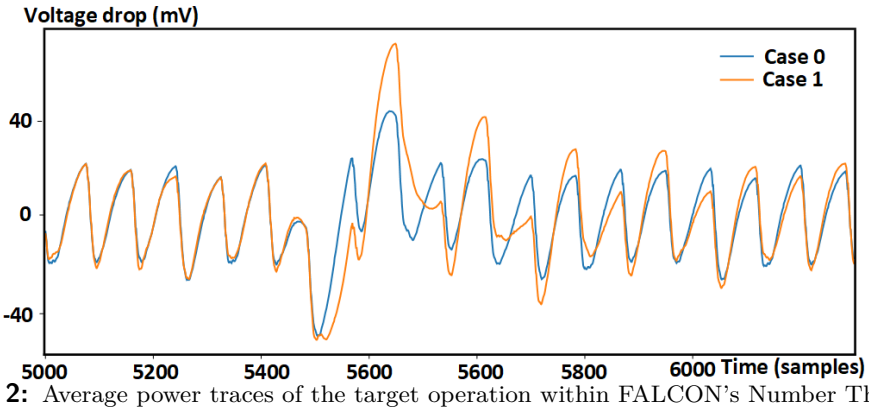


Figure 2: Average power traces of the target operation within FALCON's Number Theoretic Transform (NTT) under varying inputs collected using the experimental setup described in Subsection 5.1. The blue trace represents power consumption when no in-place modular reduction occurred, while the orange trace corresponds to instances where a reduction took place. A clear difference in power amplitude between the two cases reveals data-dependent leakage.

and by Assael et al., who employed a technique similar to belief propagation on an improved NTT implementation using Plantard modular arithmetic [AEVR23].

We argue that our approach advances BP-based attacks along two key dimensions. First, our method yields *deterministic* indications of correct key hypotheses. The recovered reduction flags directly constrain the intermediate values and propagate as hard constraints, eliminating the numerical instability that may arise from soft probabilistic updates. Second, our method substantially reduces the profiling requirements. Prior BP-based attacks require profiling nearly all possible operand-twiddle factor combinations, resulting in on the order of 10^8 profiling traces[PPM17]. In contrast, our attack exploits only two distinct leakage behaviors, corresponding to the reduction outcomes '0' and '-1'. As a result, the leakage model can be reliably obtained with as few as 2,500 traces.

Threat model We follow the well-established threat model for single-trace side-channel attacks [KAA21, GMRR22, ATT+18b, BCM+25, LZY+25, BJL+14, PPM17], assuming an adversary who has physical access to the target device. The adversary is assumed to possess knowledge of the executing software, to measure the device's power consumption, to approximate the timing of specific computations, and to intercept communication channels to capture exchanged public messages. Our attack uses a profiling phase. During this phase, the attacker can supply random inputs and analyze the software's power consumption behavior using known values. However, at runtime, the adversary is restricted to capturing a single power trace in an attempt to deduce the entire secret key. Our experiments are conducted on the STM32F417 development board, which integrates an ARM Cortex-M4 processor. We provide a detailed description of our setup in Subsection 5.1.

We provide a brief overview of countermeasures in Section 6 and focus primarily on attack methodologies, following the approach taken in prior publications [DG25, AHC+25, RPJ+24, KA21, KAA21]. The implementations of countermeasures are outside the scope of this paper. Other attacks, such as fault injection [LTYZ25, MMA25a, MMA25b], cold boot attacks [ADP18], are likewise considered out of scope.

3 Foundational Principles

In this section, we begin by demonstrating the vulnerability of the target operations. We then highlight the central role of the NTT within FALCON, motivating why leakage in this subroutine has a system-wide impact. Next, we show how the observed leakage can be exploited to drastically shrink the input space of a single butterfly operation. Finally,

we demonstrate that when these leakages are aggregated across the entire NTT structure, the brute-force search space is further reduced, leading to a substantial decrease in the overall computational effort required to mount the attack.

Leaky operation This subsection describes how negating the '31-bit right-shift' operation can lead to only two possible results and the high distinguishability of these two possible results by Hamming weight (HW).

For a 32-bit variable, the '31-bit right-shift' operation (expressed as '$(x \gg 31)$' in software) shifts the most significant bit (MSB) to the least significant bit (LSB) and clears all other bits. This operation produces only two possible outcomes: '0' or '1'. Negating the result yields either '0' or '-1', which are represented in 32-bit two's complement as all 0s and all 1s, respectively. When the processor writes the result, the '-1' case exhibits a Hamming weight (HW) of 32, while the '0' case has an HW of 0. Figure 2 presents the average power trace collected using the setup described in Subsection 5.1. It shows that **the '-1' case results in higher power consumption compared to the '0' case.** This leakage mechanism can be traced back to the established relation between Hamming weight and power consumption [KJJ99], the large HW difference observed when negating 0 versus 1 [BB56], as well as the leakage induced by right-shift operations producing 0/1 intermediates [And05]. This leakage was recently leveraged by Karabulut et al. [KAA21]. Guerreau et al. [GR24] extended the analysis of this leakage, Qiu et al used this for *another* subroutine [QA25].

In contrast to prior work, which targets other components of FALCON, *one of* our key contribution is to uncover a vulnerability in an entirely different subroutine. Specifically, the number theoretic transform (NTT). The found vulnerability, however, doesn't lead to a direct exposure to the keys as in prior work. Therefore, *another* major contribution of this work is to introduce a novel algorithm tailored for FALCON NTT implementation to aggregate the small leakages into a key recovery.

FALCON's NTT usage FALCON employs the NTT in multiple steps, including signature generation, key generation, and signature verification. During both the public key generation and signature generation phases, the NTT is applied directly to the secret lattice basis components for efficiency.

Listing 1 illustrates how FALCON employs the NTT during the signature generation process. Each time a new signature is produced, the scheme recomputes the lattice component `G`. This design decision stems from the high memory cost of storing `G` and the relatively low computational overhead of recomputing it from `f`, `g`, and `F`. Since this recomputation involves multiple polynomial multiplications, FALCON performs these operations in the NTT domain to improve efficiency. As shown in Line 7 of Listing 1, the secret lattice components are passed into the vulnerable subroutine. The input arrays `f` and `g` contain coefficients sampled from a discrete Gaussian distribution centered at zero. At Lines 20 and 27, the function `mq_conv_small()` maps all negative coefficients in these arrays to their positive modular counterparts, producing values in the range $[0, q]$ (highlighted in yellow). For instance, coefficients originally within $[-m, -1]$ are mapped to $[q - m, q - 1]$. Because of the in-place modular reduction, an adversary can infer which coefficients underwent this transformation (as shown in line 35). Finally, the processed secret polynomials are passed to the vulnerable `mq_NTT` routine.

In the key generation process, FALCON also performs computations in the NTT domain to maintain scalability for large-degree polynomials. Listing 2 shows how the secret polynomials propagate into the NTT subroutine during public-key generation. Here, the overall mechanism mirrors that of signature generation: the lattice-basis polynomials are first processed by the `mq_conv_small()` function (highlighted in yellow at lines 7 and 8), which maps negative coefficients to their modular equivalents within $[0, q]$. The same

Listing 1: FALCON's NTT Routine Call in Signature Generation Function

```
1  int crypto_sign(unsigned char *sm, unsigned long long *smlen,
2      const unsigned char *m, unsigned long long mlen,
3      const unsigned char *sk){
4      ...
5      if (u != CRYPTO_SECRETKEYBYTES) {
6          return -1;}
7      if (!Zf(complete_private)(G, f, g, F, 9, tmp.b)) {
8          return -1;}
9      ...
10     randombytes(nonce, sizeof nonce);
11     inner_shake256_init(&sc);
12     ...
13     return 0;}
14
15 int Zf(complete_private)(int8_t *G,
16     const int8_t *f, const int8_t *g, const int8_t *F,
17     unsigned logn, uint8_t *tmp){
18     ...
19     for (u = 0; u < n; u ++) {
20         t1[u] = (uint16_t)mq_conv_small(g[u]);
21         t2[u] = (uint16_t)mq_conv_small(F[u]);
22     }
23     mq_NTT(t1, logn);
24     mq_NTT(t2, logn);
25     ...
26     for (u = 0; u < n; u ++) {
27         t2[u] = (uint16_t)mq_conv_small(f[u]);
28     }
29     mq_NTT(t2, logn);
30     ...
31     return 1;}
32 static inline uint32_t mq_conv_small(int x){
33         uint32_t y;
34         y = (uint32_t)x;
35         y += Q & -(y >> 31);
36         return y;}
```

bit-shift leakage allows an adversary to determine which coefficients were reduced in this way. Subsequently, the polynomials are transformed by the vulnerable `mq_NTT` function (highlighted in red at lines 10 and 11), completing the forward NTT process that underlies the computation of the public key.

The leaky in-place modular reduction Listing 3 shows FALCON's NTT implementation. This subroutine takes input as an array of n secret coefficients and a fixed integer $\log n$ (equal to 9 for FALCON-512). The transformation is applied in place, operating directly on the input coefficients without allocating additional memory. The NTT operation consists of multiple stages, each containing several groups. Within the same stage and group, all butterflies share the same twiddle factor. This subroutine implements the scheme using a three-layer nested loop structure: each iteration of the outermost loop corresponds to a stage, each iteration of the middle loop corresponds to a group within that stage, and each iteration of the innermost loop corresponds to a single butterfly operation within that group. As described in Section 2, the subroutine follows a decimation-in-frequency (DIF) scheme,

Listing 2: FALCON's Public Key Computation Function

```
1  int Zf(compute_public)(uint16_t *h,
2      const int8_t *f, const int8_t *g, unsigned logn, uint8_t *tmp
           ){
3      size_t u, n;
4      uint16_t *tt;
5      ...
6      for (u = 0; u < n; u ++) {
7           tt[u] = (uint16_t)mq_conv_small(f[u]);
8           h[u] = (uint16_t)mq_conv_small(g[u]); }
9
10     mq_NTT(h, logn);
11     mq_NTT(tt, logn);
12     ...
13     return 1;}
```

in which the number of groups increases with the stage number, while the group size decreases accordingly. Additionally, the index-reversal step typically used in Cooley-Tukey implementations is omitted. Lines 16–18 of Listing 3 implement the core component of a butterfly operation (highlighted in red). The second input is first multiplied by the twiddle factor to produce an intermediate variable. The two outputs are then computed as the sum and the difference between the first variable and this intermediate value. It is worth noting that all three of these operations are done in modular arithmetic, meaning the results will be mod q before they are used by the next step. In the following, we show how the subroutines `mq_montymul()`, `mq_add()`, and `mq_sub()` leak critical intermediate modular reduction flags, enabling an adversary to drastically reduce the key's search space.

Listing 4 shows the implementation of FALCON's twiddle factor multiplication, addition, and subtraction within a single butterfly operation. For the multiplication step (Lines 1–8), FALCON operates in the Montgomery domain to improve efficiency by multiplying coefficients with twiddle factors from the `GMb[]` matrix, which are specifically precomputed for Montgomery-domain representation. The result is then converted back to the normal domain using a bit-shift operation (highlighted in red). It is important to note that Montgomery multiplication improves computational efficiency without altering the final numerical result. Values computed using this method are equivalent to those obtained via standard modular multiplication.

At Lines 6–7, the code applies an in-place modular reduction to map any results outside the range $[0, q-1]$ back into this range. This is achieved by applying a mask to the intermediate result z, determined by the most significant bit (MSB) of the value $z-q$. This implementation is constant-time, thereby mitigating timing side-channel attacks. However, as discussed in Section 3, negating a right-shifted 31-bit value introduces a measurable difference in power consumption, thereby revealing whether reduction occurred. Similarly, at lines 12–13 and lines 18–19, the code applies analogous in-place modular reduction techniques to map the intermediate results of the addition and subtraction operations back into the range $[0, q-1]$, thereby revealing the corresponding in-place modular reductions.

Combining the leakages While the previous subsections identified leakage points, each exposes only partial information about the underlying computation. The central challenge lies in aggregating these fragmented leakages to enable full recovery of the secret variable. This subsection addresses that challenge by presenting a method to systematically combine partial observations into a complete key reconstruction.

To take advantage of the partial information leaked by the in-place modular reduction

Listing 3: FALCON NTT subroutine

```
static void mq_NTT(uint16_t *a, unsigned logn){
    size_t n, t, m;
    n = (size_t)1 << logn;
    t = n;
    for (m = 1; m < n; m <<= 1) {
        size_t ht, i, j1;
        ht = t >> 1;
        for (i = 0, j1 = 0; i < m; i++, j1 += t) {
            size_t j2;
            uint32_t s;
            s = GMb[m + i];
            j2 = j1 + ht;
            for (j = j1; j < j2; j++) {
                uint32_t u, v;
                u = a[j];
                v = mq_montymul(a[j + ht], s);
                a[j] = (uint16_t)mq_add(u, v);
                a[j + ht] = (uint16_t)mq_sub(u, v); }}
        t = ht;}}
```

flags, a naive way would be to perform an exhaustive search over all possible input combinations and preserve the input combinations that satisfy the in-place modular reduction flags for all the stages. However, such a brute-force strategy quickly becomes infeasible: For instance, recovering FALCON-512 in this manner would require checking $(256 \times 12289 \times 12289)$ combinations per stage $\times 9$ stages $= 173,974,488,192$ combinations. Our approach overcomes this barrier by introducing a structured pruning mechanism that exploits the staged nature of the NTT. Instead of exploring the entire brute-force search space in a single step, we confine enumeration to individual butterfly stages and leverage in-place modular reduction flags to *prune* inconsistent candidates immediately. We call it the *forward pruning process*. Furthermore, we introduce the backward inference paths that iteratively refine the candidate sets based on updated output values obtained from forward pruning. We refer to it as the *backward pruning process*. Together, these techniques transform what would otherwise be an intractable exhaustive search into a scalable and deterministic attack framework capable of recovering the key from a single trace. We provide detailed, code-level implementation in Section 4.

Our pruning mechanism in a single NTT butterfly is applied in the following steps. First, during the twiddle-factor multiplication step, candidate input values are multiplied by the corresponding twiddle factor, and their in-place modular reduction behavior (i.e., whether reduction occurred) is compared against the observed side-channel measurements. Only input pairs that produce a result matching the in-place modular reduction flags are retained. These surviving pairs are then propagated to the addition and subtraction steps, where the same procedure is applied: results are checked against the measured reduction patterns, and any input-output pair failing to align is eliminated.

The input-output range reductions achieved within individual butterflies can be efficiently combined across NTT stages. Owing to the transform's intrinsic structure, where the outputs of one stage serve as the inputs to the next, these reductions naturally propagate forward. As a result, an attacker can apply the same pruning strategy stage-by-stage, passing only the surviving candidates to subsequent stages, analogous to the inference process within a single stage. This cumulative effect significantly decreases the computational effort required to mount the attack and results in a cascade of eliminations across the butterfly operation, dramatically shrinking the set of viable guesses at each

Listing 4: FALCON's buterfly implementation

```
static inline uint32_t mq_montymul(uint32_t x, uint32_t y){
    uint32_t z, w;
    z = x * y;
    w = ((z * Q0I) & 0xFFFF) * Q;
    z = (z + w) >> 16;
    z -= Q;
    z += Q & -(z >> 31);
    return z;}

static inline uint32_t mq_add(uint32_t x, uint32_t y){
    uint32_t d;
    d = x + y - Q;
    d += Q & -(d >> 31);
    return d;}

static inline uint32_t mq_sub(uint32_t x, uint32_t y){
    uint32_t d;
    d = x - y;
    d += Q & -(d >> 31);
    return d;}
```

stage. We present a detailed algorithmic-level description in Section 4 and pruning results in Section 5.

4 Attack Methodologies

This section presents our proposed attack strategy for inferring the secret polynomials in FALCON. We first provide a detailed explanation of the inference methodology used to recover the secret coefficients. We then describe the practical implementation of this method, outlining key design choices and optimizations made to ensure both accuracy and efficiency. Since prior work provides established techniques for POI extraction [Wel47, KAA21, JD23, QA25], we omit these details and instead focus on the custom method we developed to efficiently recover the secret key.

Peel the NTT: Our guess and prune (GP) method We propose the *first* guess-and-prune strategy to recover FALCON's secret input polynomial using side-channel information. Our attack implementation takes, as input, the reduction flags extracted from each butterfly operation and outputs the surviving key candidates. The approach draws inspiration from established cryptanalytic techniques such as algebraic cryptanalysis [Bar09], the extend-and-prune method [Pou25], and divide-and-conquer pruning [CPM07]. However, to the best of our knowledge, this is the first work to introduce a combined forward-enumeration and backward-consistency pruning strategy explicitly tailored to the butterfly structure of the NTT. While constraint-solving and pruning techniques like extend-and-prune have been explored in other domains of cryptanalysis, our method differs in two crucial ways: it directly exploits arithmetic in-place modular reduction flags as the pruning condition, and it systematically couples forward exploration with backward propagation across butterfly dependencies. No prior study has demonstrated such an application of pruning-based inference to reduction-flag leakage in lattice-based schemes such as FALCON. This contribution therefore introduces a new, structured framework for exploiting fine-grained arithmetic behaviors within the NTT, enabling practical and effective single-trace side-channel inference.

Algorithm 2 Pseudo Code of Our Proposed Forward Pruning

1: **Input:** Initial candidate ranges, twiddles `GMb[]`, side-channel flags $[stage, bfly]$
2: **Output:** Pruned ranges at each stage
3: **for** $stage = 0 \rightarrow 8$ **do** ▷ Iterate over NTT stages
4: **for** each butterfly $bfly$ in stage **do** ▷ Process each butterfly independently
5: `(a_idx, b_idx)` ← indices of $bfly$ ▷ Locate input indicies for this butterfly
6: $w \leftarrow$ `GMb[twiddle index(stage, bfly)]` ▷ Retrieve twiddle factor
7: (twN, addN, subN) ← Flags[stage, bfly] ▷ retrieve normalization flags
8: $S \leftarrow \emptyset$ ▷ Set of surviving outputs for this butterfly
9: **for** each $a1 \in$ `[a_idx]`, $a2 \in$ `[b_idx]` **do** ▷ Enumerate candidate inputs
10: $tw \leftarrow (a2 \cdot w) \bmod q$
11: $f_{tw} \leftarrow [tw \neq a2 \cdot w]$ ▷ twiddle multiplication reduction result
12: **if** $f_{tw} \neq twN$ **then**
13: **continue** ▷ Prune input if twiddle flag mismatches
14: **end if**
15: $a1' \leftarrow (a1 + tw) \bmod q$
16: $f_{add} \leftarrow [a1' \neq a1 + tw]$ ▷ Did addition reduce?
17: $a2' \leftarrow (a1 - tw) \bmod q$
18: $f_{sub} \leftarrow [a2' \neq a1 - tw]$ ▷ Did subtraction reduce?
19: **if** $f_{add} = addN$ **and** $f_{sub} = subN$ **then**
20: $S \leftarrow S \cup \{(a1', a2')\}$ ▷ Keep outputs only if both flags match
21: **end if**
22: **end for**
23: $Out[a_idx] \leftarrow \{a1' \mid (a1', \cdot) \in S\}$ ▷ Propagate surviving $a1'$ values
24: $Out[b_idx] \leftarrow \{a2' \mid (\cdot, a2') \in S\}$ ▷ Propagate surviving $a2'$ values
25: **end for**
26: $In \leftarrow Out$ ▷ Use survivors as input to next stage
27: **end for**

In the context of a stage of NTT butterfly operations, we define its predecessor as the immediately preceding stage that supplies its input values, and the successor as the stage that consumes its output values. For each butterfly, input-output combinations that conform to the observed reduction flags are referred to as surviving key guesses, whereas combinations that violate these flags are classified as non-surviving key guesses.

In stages that have both predecessors and successors, eliminating non-surviving key guesses removes both the inputs and outputs of that stage. Output elimination is straightforward—such values are simply not propagated to subsequent stages. Input elimination, however, requires tracing backward to the corresponding predecessor stages and removing all related input-output pairs. Consequently, our attack framework integrates both a forward inference path, which propagates constraints toward later stages, and a backward inference path, which propagates constraints toward earlier stages. A complete attack first performs forward inference from the first to the last stage, followed by backward inference to propagate elimination effects from later stages back to earlier ones.

Implementation of the Guess-and-Prune method Algorithm 2 shows the forward inference pseudocode. The variable *stage* notates the inference stage, matching the staged structure of NTT. Whereas variable *bfly* notates each of butterfly we infer within the NTT structure. Variables *a1, a2, a1', a2'* denotes the inputs and outputs of the current stage.

The algorithm begins by assigning an initial set of candidate values to all secret polynomial coefficients. For each stage of the NTT, every butterfly operation is examined independently (line 4, Algorithm 2). Within a butterfly, all candidate input pairs are multiplied by the corresponding twiddle factor, and the reduction behavior is compared

against the measured in-place modular reduction flag. Any input pair inconsistent with this observation is immediately discarded (lines 5 - 13, Algorithm 2). The remaining pairs are then propagated through the addition and subtraction branches, with each branch checked against its respective in-place modular reduction flag (lines 15 - 21, Algorithm 2). Only outputs that simultaneously satisfy both checks are retained. These surviving values are stored as the butterfly's output sets and forwarded to the next stage, where they serve as pruned candidate inputs for subsequent butterflies, as demonstrated in lines 23 -24, Algorithm 2. At every step, the mapping between candidate inputs and their corresponding outputs is preserved, ensuring that the complete input-output correspondence is available for the backward inference phase.

Algorithm 3 Our Proposed Backward Refinement Across NTT Stages

1: **Input:** Forward-trace tables with columns `a1, a2, a1', a2'`; refined outputs from stage $s{+}1$
2: **Output:** Updated tables at stages $s \in \{0, \ldots, 7\}$ containing only rows consistent with later-stage constraints
3: **procedure** BACKWARDREFINEALLSTAGES
4: **for** $s = 7 \to 0$ **do** ▷ Work backwards from later to earlier stages
5: **for** $b = 0 \to 255$ **do** ▷ Process each butterfly in stage s
6: $(i_{\text{left}}, i_{\text{right}}, col) \leftarrow$ RESOLVEPRODUCERS$(s{+}1, b)$ ▷ Identify which two butterflies in stage $s{+}1$ produced this butterfly's outputs; *col* tells us whether to use column `a1` or `a2`
7: $A1p \leftarrow$ distinct values from *col* in TRACE$(s{+}1, i_{\text{left}})$
8: $A2p \leftarrow$ distinct values from *col* in TRACE$(s{+}1, i_{\text{right}})$ ▷ Collect refined outputs from the next stage that must be matched
9: $T \leftarrow$ table TRACE(s, b) with columns `a1, a2, a1', a2'` ▷ Load this butterfly's forward trace results
10: $T' \leftarrow \{r \in T \mid r[a1'] \in A1p \ \wedge \ r[a2'] \in A2p\}$ ▷ Filter: keep only rows whose outputs match the refined sets from stage $s{+}1$
11: overwrite TRACE(s, b) with T' ▷ Save the pruned trace so that earlier stages see only consistent inputs
12: **end for**
13: **end for**
14: **end procedure**

Algorithm 3 shows the backward inference procedure. It refines candidate inputs in a stage-by-stage manner, proceeding from the final NTT stages back toward the beginning. The variable s denotes the stage that the inference is running on, whereas b denotes the butterfly index. Variables *A1p* and *A2p* denote the updated distinct values obtained from later stages. At each step, it exploits two sources of information: the refined output sets obtained from the forward pruning of later stages (as shown in lines 7-8, Algorithm 3), and the explicit input-output correspondences stored in the forward traces (line 9, Algorithm 3). For every butterfly in stage s, the algorithm consults the surviving outputs from its two child butterflies in stage $s{+}1$. These refined sets determine which outputs of the current butterfly remain feasible. By filtering its forward-trace table against these constraints, the algorithm discards all input pairs that could not have produced any of the surviving outputs (line 10, Algorithm 3). The pruned tables are then saved in place, so that earlier stages can use them as authoritative input sets, as shown in line 11.

This recursive refinement continues until the first stage is reached. At that point, all reduction-flag information collected throughout the forward and backward phases has been propagated back and combined. The result is the most aggressively pruned candidate sets for the initial inputs, ensuring that only input values consistent with the entire sequence of measured reduction behaviors remain. This yields the strongest possible narrowing of

the secret polynomial coefficients.

5 Experimental Results

This section presents an analysis of the collected side-channel information and results. Starting with the introduction of our experimental setup, we first demonstrate that our attack drastically reduces the brute-force search space of the inputs to the NTT subroutine. Next, we show the evolution of the attack's effectiveness across the stages in which inference is applied. We then introduce a method for accelerating the attack using GPU computation and report the significant reduction in execution time achieved through GPU acceleration. We discuss lattice reduction methods that can be applied to further reduce the brute-force search space. Finally, we assess the security implications of the attack on FALCON.

5.1 Experimental Setup

The target platform is an ARM Cortex-M4F processor operating on an STM32F417 board, a configuration that is widely adopted for side-channel evaluation of embedded cryptographic implementations in prior work and NIST specifications [Moo, GMRR22, HCM24, RPJ+24, GR24, QWY+25, KA21, KAA21, LZY+25]. Power measurements were acquired using a PicoScope 3206D oscilloscope at a sampling rate of 250 MHz, while the ARM processor is clocked at 30MHz. We measured power consumption by inserting a Riscure/Keysight current probe (specified bandwidth: 1 MHz-1 GHz) in series with the power supply to capture power consumption. No amplification or filtering was employed. We collected 10,000 traces to ensure robustness during profiling, although empirical evaluation indicates that 2,500 traces are sufficient for profiling. After identifying the points of interest corresponding to the leakage, the selected samples were used to construct a Gaussian leakage model for inferring reduction flags.

5.2 Attack Results

Single butterfly results Table 1 presents our attack results of leveraging the in-place modular reduction flags to attack a single butterfly operation within FALCON's NTT subroutine. The experiment targets the code segment from Lines 16–18 in Listing 3, using input values that closely approximate the statistical distribution of FALCON's secret polynomial coefficients. To ensure that the findings generalize across realistic implementations, we incorporate all twiddle factors from the `GMb[]` matrix used in FALCON's Montgomery-domain NTT. The evaluation follows a dynamic analysis methodology, exhaustively enumerating all possible input combinations for each twiddle factor. At each operation where in-place modular reduction could occur, we record whether the reduction took place and tally the number of inputs falling into the "reduced" versus "not reduced" categories. The *majority value* is defined as the reduction-flag value that occurs most frequently. Correspondingly, the *majority proportion* is defined as the fraction of inputs yielding this dominant outcome across all trials. The computed majority values and majority proportions are summarized in Table 1.

The data in the third row, second column of Table 1 reports the distribution of reduction outcomes for the twiddle-factor multiplication step. In approximately 96% of cases, the multiplication results do not require reduction, indicating a strong skew toward "no reduction" in this stage. The four columns on the right reflect the reduction behavior during the addition and subtraction steps. For this, we factor in the reduction outcome of the preceding multiplication operation. We report the percentage corresponding to the larger of the two categories within each group. Our results show a roughly balanced distribution: in about 52% of cases, the addition or subtraction results fall into the

Table 1: Single butterfly output summary of majority decision and majority proportion for multiplication, addition, and subtraction operations.

	Mul	**Add | Mul=0**	**Add | Mul=1**	**Sub | Mul=0**	**Sub | Mul=1**
Majority Value	0	0	1	1	0
Majority Proportion	96%	52.50%	52.40%	52.30%	52.10%

"reduced" category, while the remaining 48% do not.

When the number of reduced and non-reduced outcomes is statistically well-separated and evenly distributed, the reduction flags serve as highly informative side-channel signals. A more uniform split enables stronger statistical discrimination between valid and invalid key guesses, thereby increasing the pruning power at each butterfly and allowing for more effective elimination of incorrect candidates. In contrast, a highly imbalanced distribution would weaken the utility of the flag by biasing the reduction behavior toward one category, reducing its filtering capacity. Therefore, this observed balance at the addition and subtraction stages plays a crucial role in enabling search space reduction.

Full result on FALCON We present attack results on the full FALCON-512 scheme in this sub-section. The effectiveness of the attack is quantified by comparing the number of possible coefficient guesses before and after inference. Specifically, we report the percentage of surviving key guesses after each stage. To evaluate stage n, we first apply n stages of forward inference, followed by $n-1$ stages of backward inference. The remaining candidate set is then compared against the full original candidate set count to measure the reduction achieved. When all stages of the NTT are exploited, the surviving set of candidates is minimized, representing the most aggressive reduction achievable by the attack.

Table 2: Average surviving input counts and reduction ratios across NTT stages.

Number of stages inferred	0	2	3	4	5	6	7	8	9
Surviving input counts	10752	4448	3989	3557	3072	2409	1878	1348	763
Surviving counts / Original counts	100%	41.3%	37.1%	33.1%	28.5%	22.4%	17.5%	12.5%	7.1%

Table 2 summarizes the results obtained using the GP method. For practicality, in the GP evaluation stage, we limit the input space between $[-10, 10]$, since over 99.99999% of the coefficients will be sampled in this range. The first row reports the number of NTT stages included in the experiment, the second row lists the count of surviving input combinations after pruning, and the third row gives the ratio of surviving inputs relative to the total number of possible inputs. It is worth noting that FACON's secret polynomials have 512 coefficients, corresponding to approximately $512/(512 \times 21) = 4.76\%$ of the total number of input counts. Therefore, a reduction to 0% is not to be expected. Since backward inference requires at least two stages of forward inference results to propagate constraints effectively, we omit reporting values after only a single stage. Similarly, the number of surviving inputs at stage 0 corresponds to the full set of possible coefficients prior to any pruning and is shown for reference only.

Our results show that, on average, after applying inference across all nine stages of the NTT, the candidate set of coefficients is drastically reduced. The first two stages contribute the largest pruning effect, while the reduction rate slows somewhat between stages 2-4. Nevertheless, the subsequent stages consistently provide enough information to continue eliminating infeasible candidates. At the fourth stage, 33.1% of the total number of coefficients remains (3557 surviving combinations). By the ninth and final stage, fewer than 10% of the original combinations survive (763 total), and the majority of coefficients collapse to a single possible value, **with only about one in ten coefficients on average**

retaining multiple candidates. We compute residual space by exhaustively including all the possible combinations. Although this residual space, on the order of $2^{99.8}$, is still too large for direct brute force, the remaining ambiguity can be efficiently resolved using lattice reduction techniques such as LLL or BKZ, thereby reducing the search space so it become feasible for a brute-force attack. We demonstrate this process in Subsection 5.6.

The first two NTT stages yield the largest pruning because they act on compact, weakly diffused inputs and impose nearly independent constraints. With twiddles that are identity or small in effect, multiplication does not yet spread values across $\mathbb{Z}_q$, so add/sub operate in a low-wrap regime and partition the input pair space cleanly, often exceeding 1 bit of effective information per butterfly. By contrast, later stages see inputs already diffused by nontrivial twiddles, making residues closer to uniform; constraints become more *correlated* across butterflies and are applied to a hypothesis set inflated by marginal over-approximation (as referred in Subsubsection 5.6.1), so each additional stage removes a smaller fraction. This explains the disproportionately large pruning in the first two stages.

5.3 GPU Acceleration

The structure of the NTT lends itself naturally to parallelization: both the exhaustive enumerations within a single butterfly and the independent butterfly operations within the same stage can be executed concurrently. This property makes GPU acceleration an especially effective strategy for scaling the attack.

We optimized the enumeration of key guesses within each butterfly operation for GPU acceleration. The implementation was built using CuPy on an NVIDIA RTX-3090 with 10496 CUDA cores, and its performance was compared against a standard nested for-loop implementation on the CPU. At the first stage of inference, only $2 \times 11 = 22$ coefficient guesses are considered per butterfly. However, as the computation progresses into deeper stages, the number of intermediate values grows significantly due to the butterfly structure. In particular, the twiddle-factor multiplication maps this limited set of inputs into a much larger space spanning $[0, q]$, causing the candidate set to expand rapidly with stage depth. While backward propagation eventually reduces these expanded sets back to the original possible input values, the forward computation creates an explosion in intermediate states that must be evaluated.

Table 3: Runtime comparison before and after GPU acceleration.

Stage Index	**0**	**1**	**2**
CPU Runtime	71.68 ms	4285.44 ms	17600000 ms
GPU Runtime	186.88 ms	192 ms	3115.52 ms

Table 3 reports the runtime across stages for both CPU and GPU implementations. Because the GPU implementation incurs additional overhead from array-level manipulations, it does not provide an advantage when the input set is small, as seen in the first stage. However, once the input space expands, GPU parallelism offers dramatic benefits. Starting from the second stage, the GPU achieves a 22.3× speedup compared to the CPU baseline, and by the third stage, the improvement exceeds 3450×. These results demonstrate that GPU acceleration provides massive time savings for deeper stages of the attack. We omit results beyond stage 3 due to the prohibitive runtime of the CPU implementation.

5.4 Result Robustness to Noise

To further assess robustness under noisy conditions, we inject additive i.i.d. Gaussian noise into the measured power traces at the POIs. Concretely, we construct $\tilde{t} = t + n$ with $n \sim \mathcal{N}(0, \sigma_n^2)$. Within a given experiment run, the same noise distribution (same zero mean and standard deviation σ_n) is applied to all POIs, and we repeat the evaluation for

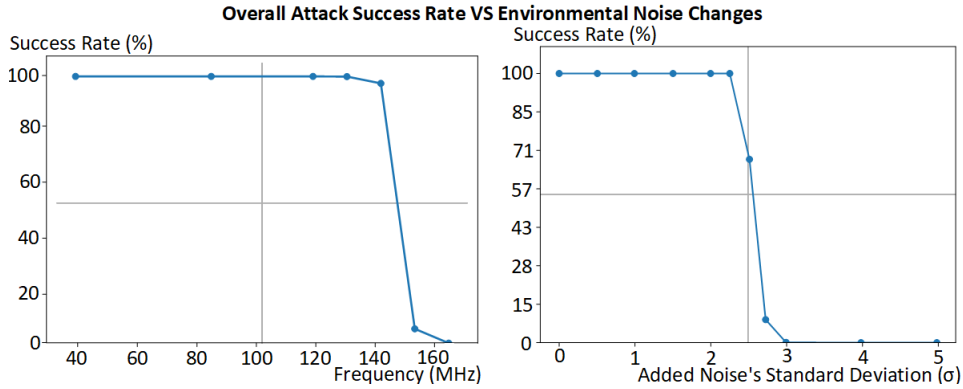


Figure 3: Attack success rate under frequency scaling and environmental noise variation. Across tested operating frequencies from 36 MHz to 168 MHz, the attack maintains a high success rate up to 132 MHz, after which performance degrades sharply. In contrast, artificially injected Gaussian noise has a more pronounced impact: the success rate remains stable at low noise levels but drops rapidly once the noise magnitude exceeds approximately 2.5× the baseline standard deviation.

different noise magnitudes by varying σ_n. We parameterize the noise level as $\sigma_n = \alpha \cdot \sigma_{\text{POI}}$, where σ_{POI} is the empirical standard deviation of the measured POI samples and α is the scaling factor shown on the x-axis. All success rates are obtained by re-running the attack on these noise-augmented traces. In addition, we evaluate the attack across different device operating frequencies, as lower frequencies yield cleaner power traces. We rerun the evaluation over a range of noise levels, and the results are summarized in Figure 3.

As shown in the figure, the attack maintains a high success rate up to 132 MHz before degrading sharply. We attribute this degradation primarily to increased environmental noise at higher operating frequencies, and secondarily to the limited effective bandwidth of the measurement setup when the operating frequency approaches the Nyquist limit. In contrast, the attack is more sensitive to additive Gaussian noise: the success rate drops once the injected noise exceeds approximately 2.5× the baseline standard deviation, revealing a clear noise threshold beyond which reliable inference is no longer possible.

5.5 The Impact on FALCON's Security

In this subsection, we evaluate the security implications of the proposed attack on FALCON. We begin with a theoretical analysis, extrapolating from the reductions observed at a single stage of the NTT. We then transition to a practical perspective, assessing the real-world impact using results obtained from our full experimental evaluation of FALCON.

5.5.1 Theoretical Impact

From Section 5, we observe that the modular reduction in the twiddle-factor multiplication produces a high-separation event: when it triggers, it discards about 96% of candidate pairs *at that step*. Because the subsequent addition and subtraction each eliminate about 50% of the remaining candidates, a triggered twiddle reduction followed by add/sub prunes the surviving space to roughly 0.02 for that butterfly (i.e., an overall ≈98% elimination within the butterfly). When the twiddle reduction does *not* trigger, the add/sub steps still provide their typical 50% cut. These numbers carry two important interpretations. First, they quantify the percentage of coefficient combinations eliminated at each step. Second, they describe the probability with which this level of separation occurs in practice. For example, in the case of twiddle-factor multiplication, roughly 96% of candidate inputs are ruled out whenever a reduction occurs. However, such a reduction happens with only about 4% probability. By contrast, the addition and subtraction steps yield a more balanced

outcome: they consistently eliminate about 50% of candidate inputs, and this occurs with equal probability for each case, effectively guaranteeing a one-half reduction on average.

For clarity, we evaluate butterflies by the fraction p of candidates they keep and measure their pruning in bits via $-\log_2 p$. This yields two regimes:

- **Strong butterflies** (twiddle reduction triggers): keep $p_{\text{strong}} \approx 0.02$ and thus contribute $-\log_2(0.02) = \log_2(50) \approx 5.6439$ bits of pruning.
- **Weak butterflies** (no twiddle reduction): keep $p_{\text{weak}} = 0.5$ and thus contribute $-\log_2(0.5) = 1$ bit.

For practicality, we limit our attack search space to [-10,+10] for each coefficient, leading to the initial FALCON-512 search space to $21^{512} = (2^{4.39})^{512} = 2^{2248}$. Across the 9-stage NTT with 256 butterflies per stage (total $N = 2304$), about $k = 0.04N \approx 92$ butterflies are strong and $N - k = 2212$ are weak (per the empirical rates above). Therefore, the total information gained is

$$I \;\approx\; (N-k)\cdot 1 \;+\; k\cdot 5.6439 \;=\; 2212 \;+\; 92\times 5.6439 \;\approx\; 2731.2 \text{ bits.}$$

Hence, the remaining search exponent is

$$Y \;=\; 2248 - I \;\approx\; -483,$$

which we conservatively clamp at $Y = 0$. In other words, the effective post-attack brute-force search space is 2^0 (unique recovery).

5.5.2 Impact Analysis Based on Real-world Results

In the real-world experiment, 763 surviving coefficients remain that satisfy all observed modular reduction flags. On average, each coefficient is left with only $763/512 = 1.5$ possible values. Since each coefficient originally had 21 possible values, our attack reduces the per-coefficient brute-force search space from $2^{4.39}$ to roughly $2^{0.59}$ on average.

From a global perspective, without any lattice reduction techniques, the total brute-force search space is reduced from 2^{2248} to approximately 2^{99}. Although this reduction alone may still require a significant amount of time for a brute-force attack. This residual uncertainty can be efficiently resolved using lattice-reduction algorithms such as LLL or BKZ, enabling an even narrower brute-force search space of the secret key in practice, which we address in the following sections. We observed a discrepancy between theoretical predictions and experimental results; a detailed explanation is provided in Subsubsection 5.6.1.

5.6 Lattice-Reduction Based Pruning with Partial f and g

Overall method This section explains how we use lattice reduction to resolve the few remaining unknown coefficients of the FALCON secret polynomials (f, g) once most of their values are already known. Starting from the public key h and partial knowledge of (f, g), we first rewrite the public-key relation so that all known contributions are removed, leaving a reduced system that depends only on the unknown coefficients. This system captures the requirement that the remaining coefficients must collectively explain the public key. We then embed this reduced system into a carefully scaled lattice and apply lattice reduction, decoding, and consistency checks, following an approach similar to [PPM17]. After each round, coefficients that become certain are fixed and fed back into the next iteration, progressively shrinking the search space until the remaining uncertainty becomes tractable.

Partial disclosure of FALCON's secret basis (f, g) leaves a small subset of coefficients undetermined. Let $n = 512$ and $q = 12289$, with

$$f, g \in \mathbb{Z}[x]/(x^n + 1), \qquad h \equiv gf^{-1} \pmod{q}.$$

Given that approximately 90% of coefficients are known, we associate each coefficient with a candidate set

$$S_f(i), S_g(j) \subset \mathbb{Z},$$

and define the unknown index sets

$$U_f = \{i : |S_f(i)| > 1\}, \qquad U_g = \{j : |S_g(j)| > 1\}.$$

Collecting the remaining unknowns yields the vector

$$x = \begin{pmatrix} f_{U_f} \\ g_{U_g} \end{pmatrix} \in \mathbb{Z}^r, \qquad r = |U_f| + |U_g|.$$

Reduced linear system Let $C(h)$ denote the convolution matrix corresponding to multiplication by the public key h. We decompose $f = f_{\text{known}} + f_{\text{unknown}}$ and $g = g_{\text{known}} + g_{\text{unknown}}$, where the unknown coefficients are indexed by U_f and U_g. The vector $p' = g_{\text{known}} - C(h)f_{\text{known}}$ is the residual obtained after subtracting the contribution of all known coefficients. The matrices $\tilde{A}$ and vector $\tilde{p}$ denote centered integer lifts of A' and p'.

Substituting the known coefficients into the public-key relation $hf \equiv g \pmod q$ yields the reduced modular system

$$A'x \equiv p' \pmod q, \tag{9}$$

where A' captures the contribution of the unknown coefficients and $p' = g_{\text{known}} - C(h)f_{\text{known}}$ represents the residual of the public-key equation after removing all known contributions.. Lifting (9) to centered representatives gives an approximate integer system

$$\tilde{A}x \approx \tilde{p},$$

whose solution corresponds to the remaining secret coefficients.

Anisotropic embedding Let $s = (s_1, \ldots, s_r)$ be positive scaling factors reflecting the uncertainty of each unknown coefficient, and let $\text{diag}(s)$ denote the corresponding diagonal matrix. We form the basis

$$B = \begin{pmatrix} q\,\text{diag}(s) & 0 & 0 \\ \tilde{A}\,\text{diag}(s) & qI_m & -\tilde{p} \\ 0 & 0 & t \end{pmatrix}, \tag{10}$$

where t is a tunable embedding parameter. The lattice $\mathcal{L}(B) = \{Bz : z \in \mathbb{Z}^{r+m+1}\}$ contains a short vector

$$v = \begin{pmatrix} qsx \\ \tilde{A}sx - \tilde{p} \\ 0 \end{pmatrix},$$

whose first r coordinates encode the unknown coefficients.

Reduction and decoding We first apply LLL to orthogonalize and stabilize the embedding basis, followed by BKZ with block size $\beta = 40$. This is a common approach taken in lattice reduction attacks:

$$B \xrightarrow{\text{LLL}} B_{\text{LLL}} \xrightarrow{\text{BKZ}_{40}} B_{\text{red}}. \tag{11}$$

From the reduced basis B_{red}, we apply Babai's nearest-plane algorithm to recover a candidate assignment. The scaling by $1/(qs)$ reverses the embedding and recovers candidate coefficient values.

$$\hat{x} = \left\lfloor \frac{1}{qs}\, v_{1:r} \right\rceil, \tag{12}$$

and evaluate its consistency via the residual

$$\rho = \|\tilde{A}\hat{x} - \tilde{p}\|_2. \tag{13}$$

This procedure is repeated across multiple embeddings (varying t and small perturbations of $\tilde{p}$), yielding a collection of candidate vectors $\{\hat{x}_i\}$.

Scoring and voting Candidates are ranked by their residuals (13), and only those satisfying $\rho_i \leq \tau$ are retained, where τ is determined adaptively from the residual distribution. For each unknown coefficient, we then perform a frequency-based vote over the surviving $\hat{x}_i$, retaining only values that appear consistently among low-residual candidates.

Iterative refinement (back-propagation) This step is referred to as back-propagation because coefficients resolved in one round are fixed and substituted back into the system to simplify subsequent rounds. After each iteration k, coefficients whose candidate sets collapse to singletons ($|S_i| = 1$) are fixed and substituted back into (9). The system is then re-embedded with updated scaling factors $s^{(k+1)}$:

$$B^{(k)} \leftarrow \mathrm{Embed}(\tilde{A}^{(k)}, \tilde{p}^{(k)}, s^{(k)}, t^{(k)}), \tag{14}$$

$$\hat{x}^{(k)} \leftarrow \mathrm{Babai}(B^{(k)}_{\mathrm{red}}), \tag{15}$$

$$S^{(k+1)} \leftarrow \mathrm{Prune}(S^{(k)}, \hat{x}^{(k)}, \tau^{(k)}). \tag{16}$$

This back-propagation step reduces the dimensionality of the lattice in subsequent rounds and yields a monotonic contraction

$$|S_f^{(k+1)}(i)| \leq |S_f^{(k)}(i)|, \qquad |S_g^{(k+1)}(j)| \leq |S_g^{(k)}(j)|.$$

In practice, the process converges after three iterations, at which point most coefficients are uniquely determined.

Reduction results On average, applying the proposed lattice-reduction framework to the partially recovered FALCON-512 instance yielded a total of 550 candidate coefficients. Among these, 500 coefficients collapsed to single, consistent values that satisfied the public-key constraints, demonstrating that the reduction phase effectively eliminated infeasible assignments and confined the residual uncertainty of f and g to a compact subspace. The remaining 12 coefficients shared 50 candidate values in total, corresponding to a final brute-force search space of approximately 2^{24} valid key hypotheses. This refinement was completed within 25 minutes on an Intel Core i7-1068NG7 CPU.

5.6.1 Analysis on Differences Between Real-world and Theoretical Results

We observe a notable gap between the theoretical impact of the vulnerability and its manifestation in practice. From a theoretical perspective, the side-channel leakage reveals more than enough information to uniquely recover all secret coefficients. In contrast, the practical execution of our attack leaves us with approximately 1.5 candidate values per coefficient on average.

We attribute this difference to the implementation choice made in our inference engine. Our inference strategy intentionally focuses on propagating only the *marginal ranges* of outputs a1' and a2' at each stage. This design greatly reduces the memory footprint, avoids disk-heavy intermediate storage, and keeps the entire pipeline lightweight enough to run efficiently in practice. The trade-off is that not preserving pairwise dependencies can admit spurious combinations in later stages. In particular, two values may both appear valid when considered independently, even though their joint occurrence was already excluded

in an earlier stage. This over-approximation slightly enlarges the candidate space, but it is manageable in practice because subsequent pruning continues to eliminate the majority of invalid values. This balance between accuracy and efficiency is critical, since storing the complete set of valid $(a1', a2')$ pairs per butterfly would require on average 8 terabytes of space per-stage and incur prohibitively slow lookup across stages.

6 Discussions

In this section, we briefly discuss potential defense methods and the application of our attack to other schemes.

Possible countermeasures As in prior studies [DG25, AHC+25, RPJ+24, KDYA25, KA21, KAA21], this work demonstrates a practical side-channel attack to highlight real-world risks associated with vulnerabilities in FALCON and to promote awareness within the developer community. We briefly outline potential countermeasures, both on the hardware and software levels, while noting that their full design is beyond the scope.

On the hardware side, constant-power architectures aim to equalize energy consumption during cryptographic operations, thereby suppressing data-dependent variations in power traces [TV04, Pla08, MRL14]. Techniques such as switched-capacitor power supplies [SG24] can isolate sensitive computations, for example, the NTT subroutine, from external current fluctuations by powering them from pre-charged capacitors, effectively reducing observable leakage. While these approaches can be highly effective, they often require substantial hardware redesign and may introduce area or performance overheads.

At the software level, hiding and shuffling are two widely adopted countermeasures against side-channel leakage. Hiding reduces data correlation in side-channel traces by inserting dummy operations, random delays, or reordering instructions [BMD22]. Shuffling, by contrast, randomizes the execution order of secret-dependent operations [Pes16, CMM17], introducing temporal noise that hinders trace alignment and degrades statistical attacks. We observe that a shuffling strategy that randomly reorders butterfly operations within the same NTT stage [RPBC20] could potentially mitigate the identified vulnerability, since butterflies within a single NTT stage operate on independent index pairs. However, this approach must be adapted to the structure and implementation details of FALCON.

Applicability to other implementations The proposed attack generalizes to implementations that negate the result of a 31-bit right shift on 32-bit intermediates or a 63-bit right shift on 64-bit secret-dependent intermediates. The identified vulnerability affects all FALCON software implementations following these operations, including both the reference and optimized versions in the NIST submission, since they share identical NTT subroutines. More generally, the leakage characteristics depend on the specific software implementation and target platform. NTT implementations used in other algorithms, such as Kyber [BDK+18], are not directly affected by this attack because their NTT implementations lack the specific operations we exploit in this work. While the underlying leakage mechanism may apply in theory, its presence and exploitability on other FPGA architectures and hardware platforms require empirical validation.

7 Conclusion

Although lattice-based cryptography offers strong post-quantum security with comparatively low computational cost, it depends on specialized operations that have received limited side-channel scrutiny. In this work, we identify a previously unknown attack in

software implementations of FALCON, stemming from the negation of values produced by a 31-bit right-shift within the Number Theoretic Transform (NTT).

Our investigation shows that this leakage reveals intermediate value assignments, making it possible to recover the full secret key. We validate the attack on a commercially available embedded platform, evaluating both the reference and optimized implementations from FALCON's NIST submission package, thereby confirming the attack's practical applicability. Importantly, the attack methodology we describe is fundamentally different from previously documented single-trace attacks and bears no resemblance to multi-trace techniques. As a result, existing countermeasures aimed at other leakage sources may not provide adequate protection and should be reassessed and adapted accordingly. This work, therefore, delivers a concrete, end-to-end attack, with the aim of emphasizing the risks of deploying FALCON's NIST-submitted implementations and raising awareness among implementers of this newly discovered side-channel vector.

Acknowledgments

This work was supported in part by the U.S. National Science Foundation under Grant No. 2241879. NCSU is an academic partner of Keysight Technologies, and we thank Keysight for their support with the tool through their Academic Partnership Program.

References

[ADP18] Martin R Albrecht, Amit Deo, and Kenneth G Paterson. Cold boot attacks on ring and module lwe keys under the ntt. *Cryptology ePrint Archive*, 2018.

[AEVR23] Guilhèm Assael, Philippe Elbaz-Vincent, and Guillaume Reymond. improving single-trace attacks on the number-theoretic transform for cortex-m4. In *2023 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*, pages 111–121. IEEE, 2023.

[AHC+25] Pierre Ayoub, Aurélien Hernandez, Romain Cayre, Aurélien Francillon, and Clémentine Maurice. Phasesca: Exploiting phase-modulated emanations in side channels. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2025(1):392–419, 2025.

[And05] Sean Eron Anderson. Bit twiddling hacks. *URL: http://graphics. stanford. edu/seander/bithacks. html*, 2005.

[ATT+18a] A. Aysu, Y. Tobah, M. Tiwari, A. Gerstlauer, and M. Orshansky. Horizontal side-channel vulnerabilities of post-quantum key exchange protocols. In *2018 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*, pages 81–88, 2018.

[ATT+18b] Aydin Aysu, Youssef Tobah, Mohit Tiwari, Andreas Gerstlauer, and Michael Orshansky. Horizontal side-channel vulnerabilities of post-quantum key exchange protocols. In *2018 IEEE international symposium on hardware oriented security and trust (HOST)*, pages 81–88. IEEE, 2018.

[Bar09] Gregory Bard. *Algebraic cryptanalysis*. Springer Science & Business Media, 2009.

[BB56] Andrew Donald Booth and Kathleen HV Booth. Automatic digital calculators. 1956.

[BCM+25] Marcus Brinkmann, Chitchanok Chuengsatiansup, Alexander May, Julian Nowakowski, and Yuval Yarom. Leaky mceliece: Secret key recovery from highly erroneous side-channel information. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2025(2):94–125, 2025.

[BDK+18] Joppe Bos, Léo Ducas, Eike Kiltz, Tancrède Lepoint, Vadim Lyubashevsky, John M Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-kyber: a cca-secure module-lattice-based kem. In *2018 IEEE European symposium on security and privacy (EuroS&P)*, pages 353–367. IEEE, 2018.

[BFM+18] Joppe W Bos, Simon Friedberger, Marco Martinoli, Elisabeth Oswald, and Martijn Stam. Assessing the feasibility of single trace power analysis of frodo. In *International Conference on Selected Areas in Cryptography*, pages 216–234. Springer, 2018.

[BHK+19] Daniel J Bernstein, Andreas Hülsing, Stefan Kölbl, Ruben Niederhagen, Joost Rijneveld, and Peter Schwabe. The sphincs+ signature framework. In *Proceedings of the 2019 ACM SIGSAC conference on computer and communications security*, pages 2129–2146, 2019.

[BJL+14] Aurélie Bauer, Eliane Jaulmes, Victor Lomné, Emmanuel Prouff, and Thomas Roche. Side-channel attack against rsa key generation algorithms. In *International Workshop on Cryptographic Hardware and Embedded Systems*, pages 223–241. Springer, 2014.

[BMD22] Martin Brisfors, Michail Moraitis, and Elena Dubrova. Side-channel attack countermeasures based on clock randomization have a fundamental flaw. Cryptology ePrint Archive, Paper 2022/1416, 2022.

[CCJ+16] Lily Chen, Lily Chen, Stephen Jordan, Yi-Kai Liu, Dustin Moody, Rene Peralta, Ray Perlner, and Daniel Smith-Tone. *Report on post-quantum cryptography*, volume 12. US Department of Commerce, National Institute of Standards and Technology, 2016.

[CHH24] Keon-Hee Choi, Jaeseung Han, and Dong-Guk Han. Single trace analysis of visible vs. invisible leakage for comparison-operation-based cdt sampling. *Electronics*, 13(23):4681, 2024.

[CMM17] Núria Costa, Ramiro Martínez, and Paz Morillo. Proof of a shuffle for lattice-based cryptography. In *Nordic Conference on Secure IT Systems*, pages 280–296. Springer, 2017.

[CPM07] Ruiliang Chen, Jung-Min Park, and Randolph Marchany. A divide-and-conquer strategy for thwarting distributed denial-of-service attacks. *IEEE Transactions on Parallel and Distributed Systems*, 18(5):577–588, 2007.

[CSM+25] Shuyi Chen, Honglin Shao, Jingdian Ming, Yuejun Liu, Yiwen Gao, and Yongbin Zhou. Serp-sca: A strengthened side-channel attack framework for falcon signature. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2025.

[DG25] Haiyue Dong and Qian Guo. Ot-pca: New key-recovery plaintext-checking oracle based side-channel attacks on hqc with offline templates. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2025(1):251–274, 2025.

[DH96] RSA Diffie-Hellman. Timing attacks on implementations of. In *Advances in CryptologyâĂŤCRYPTOâĂŹ96: 16th Annual International Cryptology Conference, Santa Barbara, California, USA, August 18–22, 1996, Proceedings*, volume 1109, page 104. Springer, 1996.

[DLP14] Léo Ducas, Vadim Lyubashevsky, and Thomas Prest. Efficient identity-based encryption over ntru lattices. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 22–41. Springer, 2014.

[EFG+22] Thomas Espitau, Pierre-Alain Fouque, François Gérard, Mélissa Rossi, Akira Takahashi, Mehdi Tibouchi, Alexandre Wallet, and Yang Yu. Mitaka: a simpler, parallelizable, maskable variant of falcon. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 222–253. Springer, 2022.

[GMRR22] Morgane Guerreau, Ange Martinelli, Thomas Ricosset, and Mélissa Rossi. The hidden parallelepiped is back again: Power analysis attacks on falcon. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pages 141–164, 2022.

[GR24] Morgane Guerreau and Mélissa Rossi. A not so discrete sampler: Power analysis attacks on hawk signature scheme. *Cryptology ePrint Archive*, 2024.

[HCM24] Suvadeep Hajra, Siddhartha Chowdhury, and Debdeep Mukhopadhyay. Estranet: An efficient shift-invariant transformer network for side-channel analysis. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2024(1):336–374, 2024.

[HSST23] Julius Hermelink, Silvan Streit, Emanuele Strieder, and Katharina Thieme. Adapting belief propagation to counter shuffling of ntts. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pages 60–88, 2023.

[JD23] Yanning Ji and Elena Dubrova. A side-channel attack on a masked hardware implementation of crystals-kyber. In *Proceedings of the 2023 Workshop on Attacks and Solutions in Hardware Security*, pages 27–37, 2023.

[JNWD24] Sönke Jendral, Kalle Ngo, Ruize Wang, and Elena Dubrova. Breaking sca-protected crystals-kyber with a single trace. In *2024 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*, pages 70–73. IEEE, 2024.

[KA98] Markus G Kuhn and Ross J Anderson. Soft tempest: Hidden data transmission using electromagnetic emanations. In *International Workshop on Information Hiding*, pages 124–142. Springer, 1998.

[KA21] Emre Karabulut and Aydin Aysu. Falcon down: Breaking falcon post-quantum signature scheme through side-channel attacks. In *2021 58th ACM/IEEE Design Automation Conference (DAC)*, pages 691–696. IEEE, 2021.

[KAA21] Emre Karabulut, Erdem Alkim, and Aydin Aysu. Single-trace side-channel attacks on ω-small polynomial sampling: with applications to ntru, ntru prime, and crystals-dilithium. In *2021 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*, pages 35–45. IEEE, 2021.

[KDYA25] Ashley Kurian, Anuj Dubey, Ferhat Yaman, and Aydin Aysu. Tpuxtract: An exhaustive hyperparameter extraction framework. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2025(1):78–103, 2025.

[KH18] Suhri Kim and Seokhie Hong. Single trace analysis on constant time cdt sampler and its countermeasure. *Applied Sciences*, 8(10):1809, 2018.

[KJJ99] Paul Kocher, Joshua Jaffe, and Benjamin Jun. Differential power analysis. In *Annual international cryptology conference*, pages 388–397. Springer, 1999.

[Kob87] Neal Koblitz. Elliptic curve cryptosystems. *Mathematics of computation*, 48(177):203–209, 1987.

[LDK+20] Vadim Lyubashevsky, Léo Ducas, Eike Kiltz, Tancrède Lepoint, Peter Schwabe, Gregor Seiler, Damien Stehlé, and Shi Bai. Crystals-dilithium. *Algorithm Specifications and Supporting Documentation*, 2020.

[LLJ+24] Jianqing Liu, Thinh Le, Tingxiang Ji, Ruozhou Yu, Demitry Farfurnik, Greg Bryd, and Daniel Stancil. The road to quantum internet: Progress in quantum network testbeds and major demonstrations. *Progress in Quantum Electronics*, 2024.

[LTYZ25] Xiuhan Lin, Mehdi Tibouchi, Yang Yu, and Shiduo Zhang. Do not disturb a sleeping falcon. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 213–244. Springer, 2025.

[LW23] Han Li and Thomas J Webster. Trends in nanomedicine. In *Nanomedicine*, pages 1–18. Elsevier, 2023.

[LZY+25] Xiuhan Lin, Shiduo Zhang, Yang Yu, Weijia Wang, Qidi You, Ximing Xu, and Xiaoyun Wang. Thorough power analysis on falcon gaussian samplers and practical countermeasure. In *IACR International Conference on Public-Key Cryptography*, pages 229–258. Springer, 2025.

[MKAA24] Arsalan Ali Malik, Emre Karabulut, Amro Awad, and Aydin Aysu. Enabling secure and efficient sharing of accelerators in expeditionary systems. *Journal of Hardware and Systems Security*, 8(2):94–112, 2024.

[MMA25a] Arsalan Ali Malik, Harshvadan Mihir, and Aydin Aysu. Craft: Characterizing and root-causing fault injection threats at pre-silicon. *arXiv preprint arXiv:2503.03877*, 2025.

[MMA25b] Arsalan Ali Malik, Harshvadan Mihir, and Aydin Aysu. Honest to a fault: Root-causing fault attacks with pre-silicon risc pipeline characterization. *arXiv preprint arXiv:2503.04846*, 2025.

[Moo] Dustin Moody. The 2nd round of the nist pqc standardization process-opening remarks at pqc 2019.

[MRL14] Matthew A Morrison, Nagarajan Ranganathan, and Jay Ligatti. Design of adiabatic dynamic differential logic for dpa-resistant secure integrated circuits. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 23(8):1381–1389, 2014.

[NCHS24] Jaesang Noh, Hyunseo Choi, Dongwoo Han, and Dong-Joon Shin. Efficient error-tolerant side-channel attacks on gpv signatures based on ordinary least squares regression. *Cryptology ePrint Archive*, 2024.

[NDGJ21] Kalle Ngo, Elena Dubrova, Qian Guo, and Thomas Johansson. A side-channel attack on a masked ind-cca secure saber kem implementation. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pages 676–707, 2021.

[Pes16] Peter Pessl. Analyzing the shuffling side-channel countermeasure for lattice-based signatures. In *International Conference on Cryptology in India*, pages 153–170. Springer, 2016.

[PFH+20] Thomas Prest, Pierre-Alain Fouque, Jeffrey Hoffstein, Paul Kirchner, Vadim Lyubashevsky, Thomas Pornin, Thomas Ricosset, Gregor Seiler, William Whyte, and Zhenfei Zhang. Falcon. *Post-Quantum Cryptography Project of NIST*, 2020.

[Pla08] André Platzer. Differential dynamic logic for hybrid systems. *Journal of Automated Reasoning*, 41(2):143–189, 2008.

[Pou25] Romain Poussier. Extend and prune side-channel attacks. In *Encyclopedia of Cryptography, Security and Privacy*, pages 852–856. Springer, 2025.

[PP19] Peter Pessl and Robert Primas. More practical single-trace attacks on the number theoretic transform. In *International Conference on Cryptology and Information Security in Latin America*, pages 130–149. Springer, 2019.

[PPM17] Robert Primas, Peter Pessl, and Stefan Mangard. Single-trace side-channel attacks on masked lattice-based encryption. In *Cryptographic Hardware and Embedded Systems–CHES 2017: 19th International Conference, Taipei, Taiwan, September 25-28, 2017, Proceedings*, pages 513–533. Springer, 2017.

[QA25] Jinyi Qiu and Aydin Aysu. Shift snare: Uncovering secret keys in falcon via single-trace analysis. *arXiv preprint arXiv:2504.00320*, 2025.

[QWY+25] Shipei Qu, Yuxuan Wang, Jintong Yu, Chi Zhang, and Dawu Gu. Trace copilot: Automatically locating cryptographic operations in side-channel traces by firmware binary instrumenting. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2025(1):128–159, 2025.

[RBL24] Tomáš Rabas, Jiří Buček, and Róbert Lórencz. Single-trace side-channel attacks on ntru implementation. *SN Computer Science*, 5:239, 2024.

[RPBC20] Prasanna Ravi, Romain Poussier, Shivam Bhasin, and Anupam Chattopadhyay. On configurable sca countermeasures against single trace attacks for the ntt: A performance evaluation study over kyber and dilithium on the arm cortex-m4. In *International Conference on Security, Privacy, and Applied Cryptography Engineering*, pages 123–146. Springer, 2020.

[RPJ+24] Prasanna Ravi, Thales Paiva, Dirmanto Jap, Jan-Pieter DâĂŹanvers, and Shivam Bhasin. Defeating low-cost countermeasures against side-channel attacks in lattice-based encryption. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2024.

[RRCB20] Prasanna Ravi, Sujoy Sinha Roy, Anupam Chattopadhyay, and Shivam Bhasin. Generic side-channel attacks on cca-secure lattice-based pke and kems. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pages 307–335, 2020.

[RSA78] Ronald L Rivest, Adi Shamir, and Leonard Adleman. A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM*, 21(2):120–126, 1978.

[SD25] Vladimir Sarde and Nicolas Debande. Improvement of side-channel attacks on mitaka. *Cryptology ePrint Archive*, 2025.

[SG24] Shreyas Sen and Archisman Ghosh. Circuit-level techniques for side-channel attack resilience: A tutorial. *IEEE Solid-State Circuits Magazine*, 16(4):96–108, 2024.

[SKL+20] Bo-Yeon Sim, Jihoon Kwon, Joohee Lee, Il-Ju Kim, Taeho Lee, Jaeseung Han, Hyojin Yoon, Jihoon Cho, and Dong-Guk Han. Single-trace attacks on the message encoding of lattice-based kems. Cryptology ePrint Archive, Report 2020/992, 2020. https://eprint.iacr.org/2020/992.

[SY23] Kiarash Sedghighadikolaei and Attila Altay Yavuz. A comprehensive survey of threshold signatures: Nist standards, post-quantum cryptography, exotic techniques, and real-world applications. *arXiv preprint arXiv:2311.05514*, 2023.

[TV04] Kris Tiri and Ingrid Verbauwhede. A logic level design methodology for a secure dpa resistant asic or fpga implementation. In *Proceedings Design, Automation and Test in Europe Conference and Exhibition*, volume 1, pages 246–251. IEEE, 2004.

[VCGS14] Nicolas Veyrat-Charvillon, Benoît Gérard, and François-Xavier Standaert. Soft analytical side-channel attacks. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 282–296. Springer, 2014.

[Wel47] Bernard L Welch. The generalization of âĂŸstudent'sâĂŹproblem when several different population varlances are involved. *Biometrika*, 34(1-2):28–35, 1947.

[ZLC+20] Cong Zhang, Zilong Liu, Yuyang Chen, Jiahao Lu, and Dongsheng Liu. A flexible and generic gaussian sampler with power side-channel countermeasures for quantum-secure internet of things. *IEEE Internet of Things Journal*, 2020.

[ZLYW23] Shiduo Zhang, Xiuhan Lin, Yang Yu, and Weijia Wang. Improved power analysis attacks on falcon. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 565–595. Springer, 2023.